

Guia Técnico: Comunicação BLE Passo a Passo (Android/Kotlin)

Este documento serve como um roteiro para explicar a implementação da camada de comunicação Bluetooth Low Energy (BLE) deste projeto para uma turma de pós-graduação, focando em boas práticas de engenharia de software e IoT.

1. Arquitetura e Orquestração (MVVM)

Antes de entrar no rádio, é preciso entender como o dado flui:

- **Responsabilidade:** O hardware é gerenciado no `BleDatasource`. O `BluetoothViewModel` observa as mudanças e a UI (Compose) apenas reage.
- **Concorrência:** Uso de **Kotlin Coroutines**. Operações de rádio são assíncronas; nunca bloqueamos a Main Thread para esperar uma resposta do Bluetooth.

2. Passo 0: Scan Seletivo

Não buscamos todos os dispositivos. Filtramos para economizar bateria e poluição na UI.

- **Implementação:** `scanCallback` no `BleDatasource.kt`.
- **Conceito:** Filtro por `Service UUID` ou nome (ESP32).
- **Destaque:** Uso do `SCAN_MODE_LOW_LATENCY` para garantir que o dispositivo seja encontrado rapidamente em ambientes ruidosos.

3. Passo 1: Negociação de MTU (Maximum Transmission Unit)

O “aperto de mão” inicial para definir o tamanho do pacote.

- **Código:** `gatt.requestMtu(64)` .
- **Por que 64?:** O padrão BLE é ~23 bytes. Nosso pacote tem 45 bytes + overhead. Definir 64 bytes garante que o pacote inteiro chegue em uma única transação, evitando fragmentação e economizando energia.

4. Passo 2: Descoberta de Serviços e Características

O Bluetooth funciona como uma árvore de arquivos (GATT).

- **Fluxo:** Conexão -> `discoverServices()` -> `onServicesDiscovered` .
- **Mapeamento:** Localizamos o `SERVICE_UUID` e o `CHARACTERISTIC_UUID` (Canal de dados) definidos no firmware do ESP32.

5. Passo 3: Habilitação de Notificações (CCCD)

Em vez do Android perguntar “tem dado?” (Polling), o ESP32 avisa “aqui está o dado” (Push).

- **Configuração:** Escrita no descritor de configuração (`00002902` . . .).
- **Importância:** É o que permite o fluxo de telemetria em tempo real sem sobrecarregar o processador do smartphone.

6. Passo 4: O Protocolo Binário (Engenharia de Dados)

A grande mudança de performance deste projeto.

- **O Problema:** JSON via BLE gerava pacotes de >300 bytes (instável e lento).
- **A Solução:** `Esp32PacketDecoder` .
- **Processamento:**
 1. Validar Cabeçalho (`0xAA 0x55`).
 2. Validar Integridade via `CRC16` .
 3. Se o CRC falhar, o pacote é corrompido e descartado antes de chegar na UI.

7. Passo 5: Telemetria e Distância (RSSI)

Transformando sinal de rádio em informação geográfica.

- **Fórmula:** Log-Distance Path Loss.
- **Dinâmica:** `gatt.readRemoteRssi()` é chamado periodicamente. O valor (ex: -65dBm) entra no cálculo com o **Fator N** e o **RSSI de Referência** para gerar a distância em metros.

8. Robustez e Ciclo de Vida

Gerenciamento de recursos críticos.

- **O “Zumbi BLE”:** Se você não chamar `gatt.close()` no encerramento, a conexão fica “pendurada”, impedindo novas conexões e drenando a bateria.
 - **Implementação:** Garantir que o `disconnect()` feche o objeto GATT de forma definitiva.
-